



A Language that Helps You to Draw Graphs

DOT is a simple, plain text language that can describe graphs which both humans and computers can use. It can be used to generate graphs in different formats such as *.ps* and *.pdf*. This article describes how to install DOT, draw a graph, and then generate a graph that depicts an execution flow of a program in C.

DOT is a language you can use to textually represent a graph, so that it can be processed by the dot tool to render the graph as a graphical representation in different formats like *.ps*, *.pdf*, etc. It has been developed at AT&T Labs as part of the Graphviz project, which is a collection of tools for graph visualisation. DOT is open source free software published under the Eclipse Public License.

Installation

For the purposes of this article I will use the Ubuntu 14.04 LTS OS version to install DOT; however, you can use any distribution you choose. The dot tool comes with the Graphviz package along with many other graph drawing tools. Run the following command to install Graphviz:

```
$ sudo apt-get install graphviz
```

Understanding graphs

Before we start drawing our first graph, let's first understand what it is, in a non-technical way. A graph *G* consists of a set of nodes called vertices and a set of edges connecting a pair of vertices. If the edges have direction, it is a directed graph; else, it is an undirected graph. So the diagram in Figure 1 is a directed graph or digraph. *v1*, *v2*, *v3* and *v4* are vertices, and *e1*, *e2* and *e3* are edges connecting the ordered vertex-pairs (*v1*, *v2*), (*v1*, *v4*) and (*v4*, *v3*), respectively. All edges have a direction.

Our first graph using DOT

Open a text editor and enter the following code to draw the graph as shown in Figure 1; save it as *firstgraph.dot*.

```
digraph G {
    v1 -> v2 [label="e1"];
    v1 -> v4 [label="e2"];
    v4 -> v3 [label="e3"];
}
```

To generate the graph in PDF format, run the following:

```
$ dot firstgraph.dot -Tpdf -o firstgraph.pdf
```

Run the following command to see the graph (Figure 2).

```
$ evince firstgraph.pdf
```

Generate an execution flow graph

Now that we know how to draw basic graphs, let's generate a graph for the execution flow of a C program. The following C program *oddeven.c* checks whether a given integer number is odd or even.

```
#include<stdio.h>
int main()
{
```